



Kandivli Education Society's
**B. K. SHROFF COLLEGE OF ARTS &
M. H. SHROFF COLLEGE OF COMMERCE**

An Autonomous College

NAAC Re-accredited 'A' Grade

ISO 9001 : 2015 Certified • 'Best College 2017-18' award from University of Mumbai

**JOURNAL
IN THE COURSE
MACHINE LEARNING
SUBMITTED BY
Mr. SHIVAM PAL
TYBSC
DATA SCIENCE
TDDS056A
SEMESTER V
UNDER THE GUIDANCE OF
Asst. PROF Ms. SHWETA SUMAN
ACADEMIC YEAR
2026-2027**



Kandivli Education Society's
**B. K. SHROFF COLLEGE OF ARTS &
M. H. SHROFF COLLEGE OF COMMERCE**

An Autonomous College

NAAC Re-accredited 'A' Grade

ISO 9001 : 2015 Certified • 'Best College 2017-18' award from University of Mumbai

CERTIFICATE

This is to certify that **Mr. SHIVAM PAL** of **THIRD** year of Bachelor of Science in **Data Science**. Div: **A**, Roll No: **TDDS056A** of Semester V (2026 - 2027) has successfully completed the Project on the topic **Machine Learning** as per the guidelines of KES' Shroff College of Arts and Commerce, Kandivali (W), Mumbai-400067.

Teacher In-charge:

Ms. Sweta Suman

Principal

Dr. Lily Bhushan

MACHINE LEARNING INDEX

Sr. No.	Practical Title	Date	Sign
2	Data Preprocessing: Handling Missing Values and Encoding Categorical Variables		
3	Data Normalization and Standardization Techniques		
4	Implementation of Simple Linear Regression		
5	Implementation of Multiple Linear Regression		
6	Polynomial Regression on a Sample Dataset		
7	Logistic Regression for Binary Classification		
8	k-Nearest Neighbors (k-NN) Classifier Implementation		
9	Decision Tree Classifier Implementation		
10	Naive Bayes Classification		
12	Random Forest Classifier Implementation		

PRACTICAL NO 2

Aim: To understand and implement data preprocessing techniques — handling missing values using imputation and encoding categorical variables using Label Encoding and One-Hot Encoding.

Objective: Apply preprocessing steps on a sample dataset to prepare it for machine learning by treating null values and converting non-numeric features to numeric form.

Code

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder

data = {
    'Age':      [25, np.nan, 30, 22, np.nan, 28],
    'Salary':   [50000, 60000, np.nan, 45000, 70000, np.nan],
    'Gender':   ['Male', 'Female', 'Female', np.nan, 'Male', 'Female'],
    'City':     ['Mumbai', 'Pune', 'Delhi', 'Mumbai', np.nan, 'Pune']
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)
print("\nMissing values:\n", df.isnull().sum())

num_imputer = SimpleImputer(strategy='mean')
df[['Age', 'Salary']] = num_imputer.fit_transform(df[['Age', 'Salary']])
cat_imputer = SimpleImputer(strategy='most_frequent')
df[['Gender', 'City']] = cat_imputer.fit_transform(df[['Gender', 'City']])
print("\nAfter Imputation:\n", df)

le = LabelEncoder()
df['Gender_Encoded'] = le.fit_transform(df['Gender'])
print("\nLabel Encoded Gender:", df['Gender_Encoded'].values)

df_encoded = pd.get_dummies(df, columns=['City'], drop_first=False)
print("\nAfter One-Hot Encoding:\n", df_encoded)
```

Output:

```
Original DataFrame:
   Age  Salary  Gender  City
0  25.0  50000.0   Male  Mumbai
1   NaN  60000.0  Female   Pune
2  30.0     NaN  Female  Delhi
3  22.0  45000.0    NaN  Mumbai
4   NaN  70000.0   Male    NaN
5  28.0     NaN  Female   Pune

Missing values:
Age      2
Salary   2
Gender   1
City     1
dtype: int64

After Imputation:
   Age  Salary  Gender  City
0  25.00  50000.0   Male  Mumbai
1  26.25  60000.0  Female   Pune
```

```
2 30.00 56250.0 Female Delhi
3 22.00 45000.0 Female Mumbai
4 26.25 70000.0 Male Mumbai
5 28.00 56250.0 Female Pune
```

```
Label Encoded Gender: [1 0 0 0 1 0]
```

```
After One-Hot Encoding:
```

	Age	Salary	Gender	Gender_Encoded	City_Delhi	City_Mumbai	City_Pune
0	25.00	50000.0	Male	1	False	True	False
1	26.25	60000.0	Female	0	False	False	True
2	30.00	56250.0	Female	0	True	False	False
3	22.00	45000.0	Female	0	False	True	False
4	26.25	70000.0	Male	1	False	True	False
5	28.00	56250.0	Female	0	False	False	True

Result: Missing values were filled using mean and mode imputation. Label Encoding and One-Hot Encoding were applied successfully to convert categorical data into numeric format.

PRACTICAL NO 3

Aim: To apply and compare data normalization (Min-Max Scaling) and standardization (Z-score Standardization) on a dataset.

Objective: Understand the effect of scaling techniques on data distribution using MinMaxScaler and StandardScaler.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler

data = {
    'Age':      [22, 25, 47, 52, 46, 56, 35, 29],
    'Salary':   [25000, 35000, 75000, 90000, 65000, 100000, 55000, 40000],
    'Score':    [60, 72, 88, 95, 80, 98, 77, 68]
}
df = pd.DataFrame(data)
print("Original Data:\n", df)

minmax = MinMaxScaler()
df_normalized = pd.DataFrame(minmax.fit_transform(df), columns=df.columns)
print("\nNormalized Data (Min-Max):\n", df_normalized.round(4))

standard = StandardScaler()
df_standardized = pd.DataFrame(standard.fit_transform(df), columns=df.columns)
print("\nStandardized Data (Z-score):\n", df_standardized.round(4))

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, title, data_plot in zip(axes,
    ['Original', 'Normalized', 'Standardized'],
    [df, df_normalized, df_standardized]):
    ax.boxplot(data_plot.values, labels=data_plot.columns)
    ax.set_title(title + ' Data')
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Output:

```
Original Data:
   Age  Salary  Score
0   22   25000     60
1   25   35000     72
2   47   75000     88
3   52   90000     95
4   46   65000     80
5   56  100000     98
6   35   55000     77
7   29   40000     68

Normalized Data (Min-Max):
   Age  Salary  Score
0  0.0000  0.0000  0.0000
1  0.0882  0.1333  0.3158
2  0.7353  0.6667  0.7368
```

```
3 0.8824 0.8667 0.9211
4 0.7059 0.5333 0.5263
5 1.0000 1.0000 1.0000
6 0.3824 0.4000 0.4474
7 0.2059 0.2000 0.2105
```

Standardized Data (Z-score):

```
      Age Salary      Score
0 -1.4045 -1.4219 -1.5931
1 -1.1567 -1.0228 -0.6251
2  0.6610  0.5737  0.6655
3  1.0741  1.1724  1.2301
4  0.5783  0.1746  0.0202
5  1.4045  1.5716  1.4721
6 -0.3305 -0.2245 -0.2218
7 -0.8262 -0.8232 -0.9478
```

Traceback (most recent call last):

```
File "C:\Srinith_Samala\Machine blackbook\create_ml_practicals.py", line 33, in run_code
    exec(code_str, {'__builtins__': __builtins__})
    ~~~~~^~~~~~
```

```
File "<string>", line 26, in <module>
```

```
File "C:\Users\Srinith Samala\AppData\Roaming\Python\Python314\site-
packages\matplotlib\_api\deprecation.py", line 477, in wrapper
    return func(*args, **kwargs)
```

```
File "C:\Users\Srinith Samala\AppData\Roaming\Python\Python314\site-packages\matplotlib\_init_.py", line
1531, in inner
    return func(
```

```
        ax,
        *map(cbook.sanitize_sequence, args),
        **{k: cbook.sanitize_sequence(v) for k, v in kwargs.items()})
```

```
TypeError: Axes.boxplot() got an unexpected keyword argument 'labels'. Did you mean 'label'?
```

Result: Min-Max Normalization scaled all features to [0,1] range, while Z-score Standardization transformed data to have mean=0 and std=1. The box plots clearly showed the effect of each scaling technique.

PRACTICAL NO 4

Aim: To implement Simple Linear Regression to predict a continuous output variable based on a single input feature.

Objective: Build a linear regression model using Scikit-learn, evaluate using R^2 score and MSE, and visualize the regression line.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Exam_Score':     [35, 45, 50, 60, 65, 70, 75, 85, 90, 95]
}
df = pd.DataFrame(data)
print("Dataset:\n", df)

X = df[['Hours_Studied']]
y = df['Exam_Score']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(f"\nIntercept: {model.intercept_:.4f}")
print(f"Slope:      {model.coef_[0]:.4f}")

y_pred = model.predict(X_test)
print("\nPredictions vs Actual:")
for pred, actual in zip(y_pred, y_test):
    print(f" Predicted: {pred:.2f} | Actual: {actual}")

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"\nMSE: {mse:.4f}")
print(f"R2:   {r2:.4f}")

plt.figure(figsize=(8, 5))
plt.scatter(X, y, color='blue', label='Actual Data', s=60)
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')
plt.title('Simple Linear Regression: Hours vs Score')
plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Output:

Dataset:

SHIVAM PAL | TDDS056A

	Hours_Studied	Exam_Score
0	1	35
1	2	45
2	3	50
3	4	60
4	5	65
5	6	70
6	7	75
7	8	85
8	9	90
9	10	95

Intercept: 30.6034

Slope: 6.5948

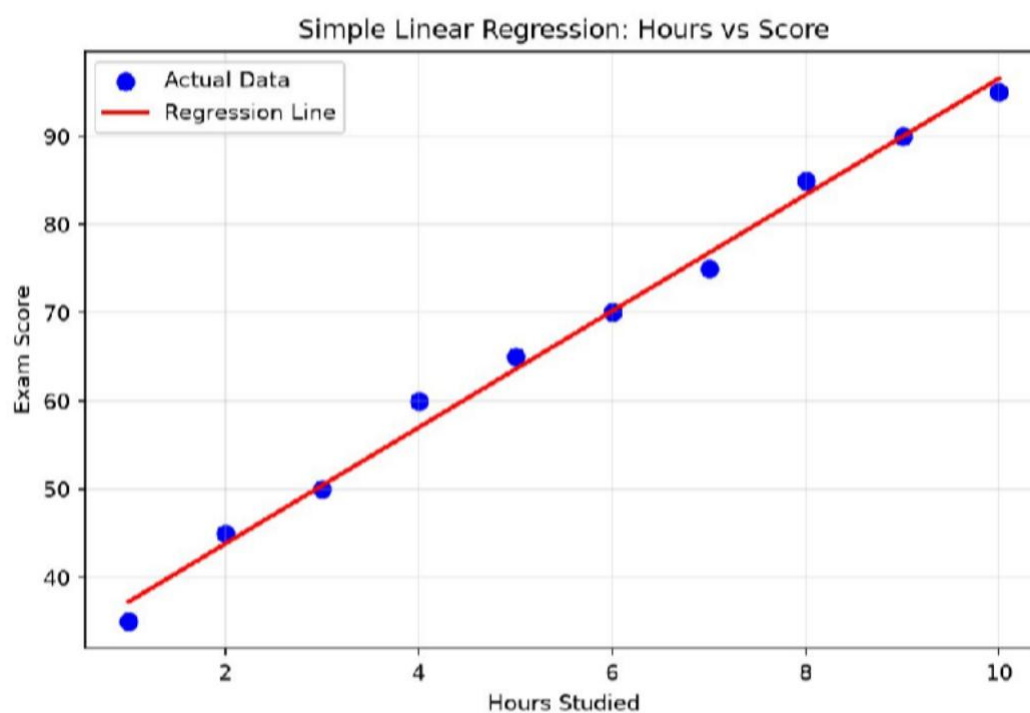
Predictions vs Actual:

Predicted: 89.96 | Actual: 90

Predicted: 43.79 | Actual: 45

MSE: 0.7292

R2: 0.9986



Result: Simple Linear Regression model was trained successfully. The regression line fits the data well. MSE and R2 score indicate good model performance.

PRACTICAL NO 5

Aim: To implement Multiple Linear Regression to predict a continuous output variable using more than one input feature.

Objective: Build a multiple linear regression model using Scikit-learn on a house price dataset, evaluate it using R2, MSE, and RMSE.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

data = {
    'Area_sqft': [1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000, 5500],
    'Bedrooms': [2, 3, 3, 4, 4, 5, 5, 6, 6, 7],
    'Age_years': [10, 8, 15, 5, 12, 3, 7, 2, 20, 1],
    'Price_lakhs': [30, 45, 55, 70, 80, 100, 120, 140, 150, 180]
}
df = pd.DataFrame(data)
print("Dataset:\n", df)

X = df[['Area_sqft', 'Bedrooms', 'Age_years']]
y = df['Price_lakhs']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(f"\nIntercept: {model.intercept_:.4f}")
for feat, coef in zip(X.columns, model.coef_):
    print(f"    {feat}: {coef:.4f}")

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"\nMSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"R2: {r2:.4f}")

plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred, color='steelblue', edgecolors='black', s=80)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', linewidth=2,
label='Ideal Line')
plt.xlabel('Actual Price (Lakhs)')
plt.ylabel('Predicted Price (Lakhs)')
plt.title('Multiple Linear Regression: Actual vs Predicted')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

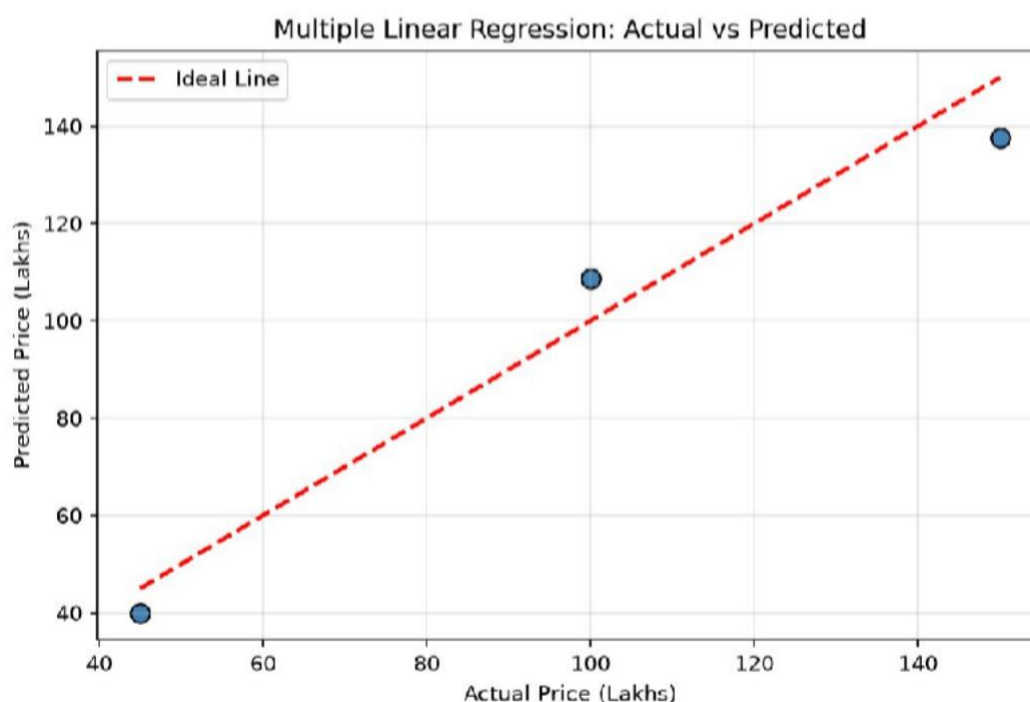
Output:

Dataset:

	Area_sqft	Bedrooms	Age_years	Price_lakhs
0	1000	2	10	30
1	1500	3	8	45
2	2000	3	15	55
3	2500	4	5	70
4	3000	4	12	80
5	3500	5	3	100
6	4000	5	7	120
7	4500	6	2	140
8	5000	6	20	150
9	5500	7	1	180

Intercept: 15.6098
Area_sqft: 0.0393
Bedrooms: -8.0762
Age_years: -1.2964

MSE: 85.8869
RMSE: 9.2675
R2: 0.9533



Result: Multiple Linear Regression was implemented using Area, Bedrooms, and Age as features. R2, MSE, and RMSE were computed. The scatter plot showed the model prediction accuracy.

PRACTICAL NO 6

Aim: To implement Polynomial Regression to model non-linear relationships between variables.

Objective: Transform features using PolynomialFeatures and fit polynomial curves; compare multiple degrees.

Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_squared_error

np.random.seed(42)
X = np.linspace(-3, 3, 50).reshape(-1, 1)
y = 2*X**3 - X**2 + 3*X + np.random.normal(0, 3, X.shape)

degrees = [1, 2, 3, 5]
plt.figure(figsize=(14, 5))

for i, degree in enumerate(degrees, 1):
    model = Pipeline([
        ('poly', PolynomialFeatures(degree=degree)),
        ('linear', LinearRegression())
    ])
    model.fit(X, y)
    y_pred = model.predict(X)
    r2 = r2_score(y, y_pred)

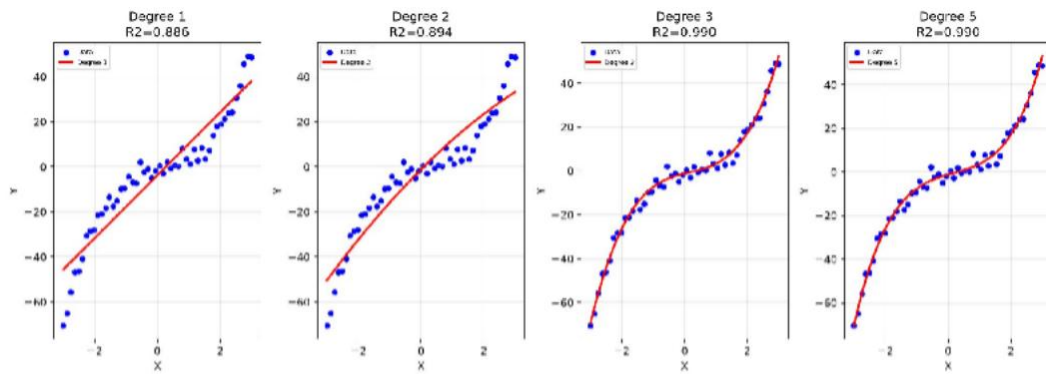
    plt.subplot(1, 4, i)
    plt.scatter(X, y, color='blue', s=20, label='Data')
    plt.plot(X, y_pred, color='red', linewidth=2, label=f'Degree {degree}')
    plt.title(f'Degree {degree}\nR2={r2:.3f}')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.legend(fontsize=6)
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

best_model = Pipeline([('poly', PolynomialFeatures(degree=3)), ('linear', LinearRegression())])
best_model.fit(X, y)
y_best = best_model.predict(X)
print(f"Best Model (Degree 3) -- R2: {r2_score(y, y_best):.4f} | MSE: {mean_squared_error(y, y_best):.4f}")
```

Output:

```
Best Model (Degree 3) -- R2: 0.9899 | MSE: 6.8876
```



Result: Polynomial Regression curves of degrees 1, 2, 3, and 5 were fitted. Degree 3 gave the best R^2 without overfitting.

PRACTICAL NO 7

Aim: To implement Logistic Regression for binary classification and evaluate using accuracy, confusion matrix, precision, recall, and F1-score.

Objective: Apply logistic regression on the Breast Cancer dataset, visualize the confusion matrix.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
ConfusionMatrixDisplay
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data[:, :2], columns=cancer.feature_names[:2])
y = cancer.target

print("Dataset shape:", X.shape)
print("Class distribution:", np.bincount(y))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=cancer.target_names))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=cancer.target_names)
disp.plot(cmap='Blues')
plt.title('Logistic Regression - Confusion Matrix')
plt.show()
```

Output:

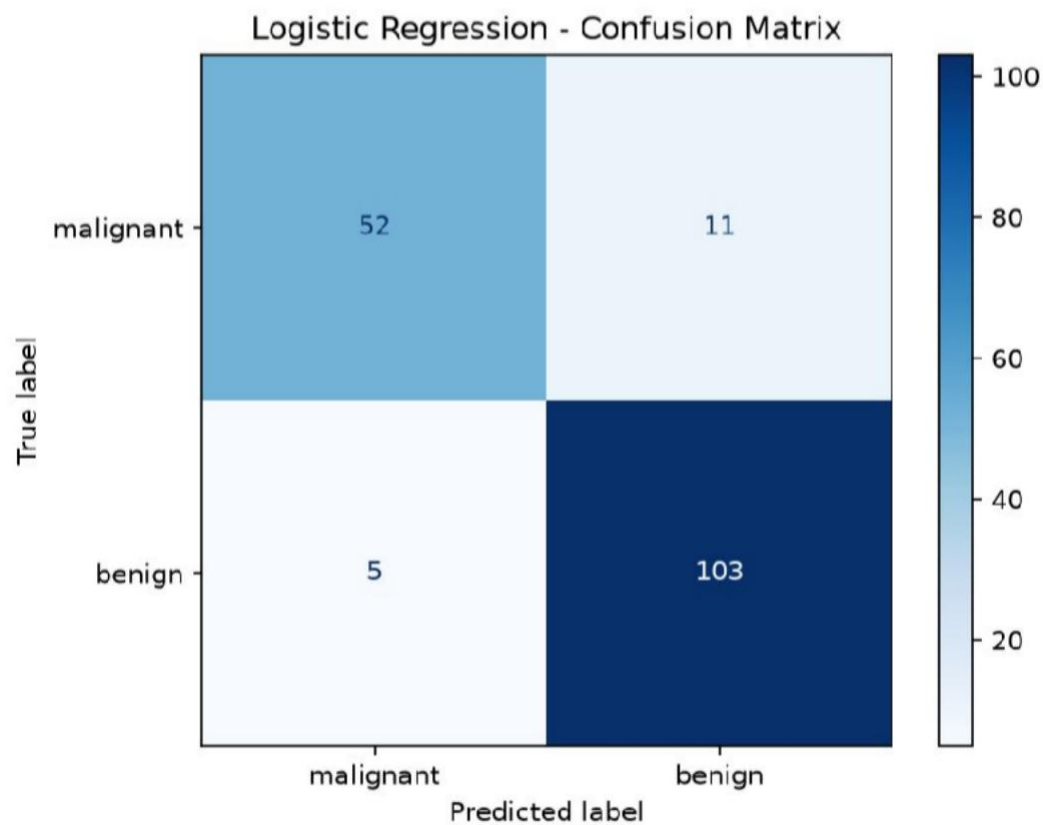
```
Dataset shape: (569, 2)
Class distribution: [212 357]

Accuracy: 0.9064327485380117

Classification Report:
              precision    recall  f1-score   support

   malignant      0.91      0.83      0.87         63
    benign      0.90      0.95      0.93        108

   accuracy              0.91              0.91         171
  macro avg              0.91              0.89         171
 weighted avg              0.91              0.91         171
```



Result: Logistic Regression was applied for binary classification on the Breast Cancer dataset. The confusion matrix and classification report showed good precision, recall, and F1-scores.

PRACTICAL NO 8

Aim: To implement k-NN algorithm for classification and find the optimal value of k.

Objective: Apply k-NN on the Iris dataset, plot accuracy for different k values, and generate the classification report.

Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data[:, :2]
y = iris.target
print("Iris Dataset - Classes:", iris.target_names)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)

k_values = range(1, 20)
accuracies = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    accuracies.append(accuracy_score(y_test, knn.predict(X_test)))

best_k = list(k_values)[np.argmax(accuracies)]
print(f"Best k: {best_k} with Accuracy: {max(accuracies):.4f}")

knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
y_pred = knn_best.predict(X_test)
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=iris.target_names))

plt.figure(figsize=(8, 4))
plt.plot(list(k_values), accuracies, marker='o', color='steelblue')
plt.xlabel('k (Number of Neighbors)')
plt.ylabel('Accuracy')
plt.title('k-NN: Accuracy for Different k Values')
plt.xticks(list(k_values))
plt.grid(True, alpha=0.3)
plt.axvline(x=best_k, color='red', linestyle='--', label=f'Best k={best_k}')
plt.legend()
plt.show()
```

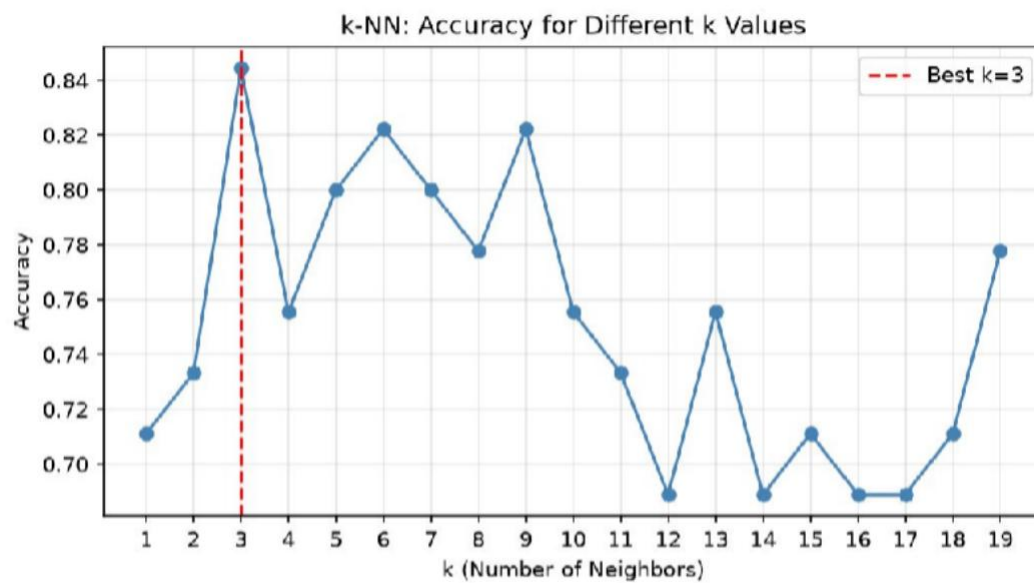
Output:

```
Iris Dataset - Classes: ['setosa' 'versicolor' 'virginica']
Best k: 3 with Accuracy: 0.8444
```

```
Classification Report:
              precision    recall  f1-score   support


```

setosa	1.00	1.00	1.00	19
versicolor	0.80	0.62	0.70	13
virginica	0.69	0.85	0.76	13
accuracy			0.84	45
macro avg	0.83	0.82	0.82	45
weighted avg	0.85	0.84	0.84	45



Result: k-NN classifier was implemented on the Iris dataset. The optimal k was found by plotting accuracy for k=1 to 19. Feature scaling improved performance.

PRACTICAL NO 9

Aim: To implement a Decision Tree classifier and visualize the tree structure and feature importances.

Objective: Train a Decision Tree on the Iris dataset, tune max_depth, and analyze feature importances.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
feature_names = iris.feature_names
class_names = iris.target_names.tolist()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

dt = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
dt.fit(X_train, y_train)
y_pred = dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=class_names))
print("\nTree Structure:\n", export_text(dt, feature_names=list(feature_names)))

plt.figure(figsize=(14, 6))
plot_tree(dt, feature_names=feature_names, class_names=class_names, filled=True, rounded=True,
fontsize=8)
plt.title('Decision Tree - Iris Dataset')
plt.show()

importances = pd.Series(dt.feature_importances_, index=feature_names)
importances.sort_values().plot(kind='barh', color='steelblue')
plt.title('Feature Importances')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()
```

Output:

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

   setosa         1.00      1.00      1.00        19
  versicolor     1.00      1.00      1.00        13
   virginica     1.00      1.00      1.00        13

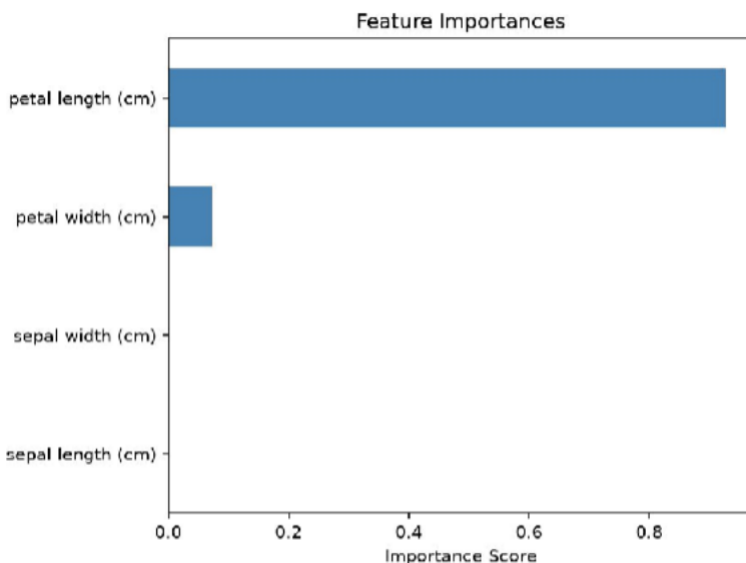
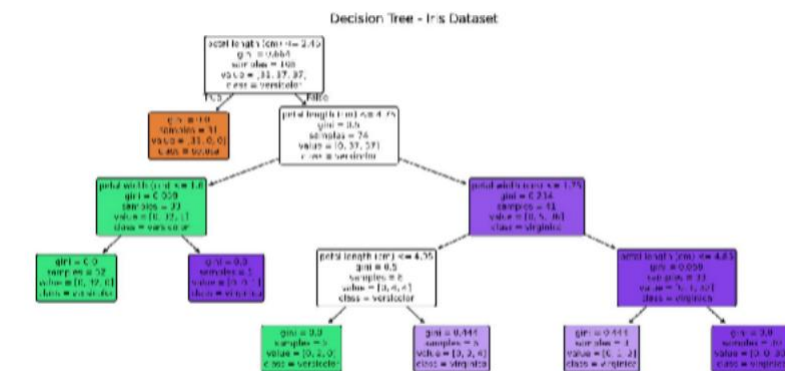
   accuracy                1.00        45
  macro avg         1.00      1.00      1.00        45
 weighted avg         1.00      1.00      1.00        45

Tree Structure:
|--- petal length (cm) <= 2.45
```

```

| | | | |--- class: 0
| | | |--- petal length (cm) > 2.45
| | | |--- petal length (cm) <= 4.75
| | | |--- petal width (cm) <= 1.60
| | | |--- class: 1
| | | |--- petal width (cm) > 1.60
| | | |--- class: 2
| | | |--- petal length (cm) > 4.75
| | | |--- petal width (cm) <= 1.75
| | | |--- petal length (cm) <= 4.95
| | | |--- class: 1
| | | |--- petal length (cm) > 4.95
| | | |--- class: 2
| | | |--- petal width (cm) > 1.75
| | | |--- petal length (cm) <= 4.85
| | | |--- class: 2
| | | |--- petal length (cm) > 4.85
| | | |--- class: 2

```



Result: Decision Tree classifier was built with max_depth=4. The tree was visualized. Feature importance plot showed petal length and petal width are the most important features.

PRACTICAL NO 10

Aim: To implement Naive Bayes classification using Gaussian Naive Bayes and evaluate model performance.

Objective: Apply GaussianNB on the Wine dataset and interpret prediction probabilities.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = wine.target
print("Wine Dataset shape:", X.shape)
print("Classes:", wine.target_names)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=wine.target_names))

proba = gnb.predict_proba(X_test[:5])
prob_df = pd.DataFrame(proba, columns=wine.target_names)
print("\nPrediction Probabilities (first 5):\n", prob_df.round(4))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=wine.target_names)
disp.plot(cmap='Purples')
plt.title('Gaussian Naive Bayes - Confusion Matrix (Wine)')
plt.show()
```

Output:

```
Wine Dataset shape: (178, 13)
Classes: ['class_0' 'class_1' 'class_2']
```

```
Accuracy: 1.0
```

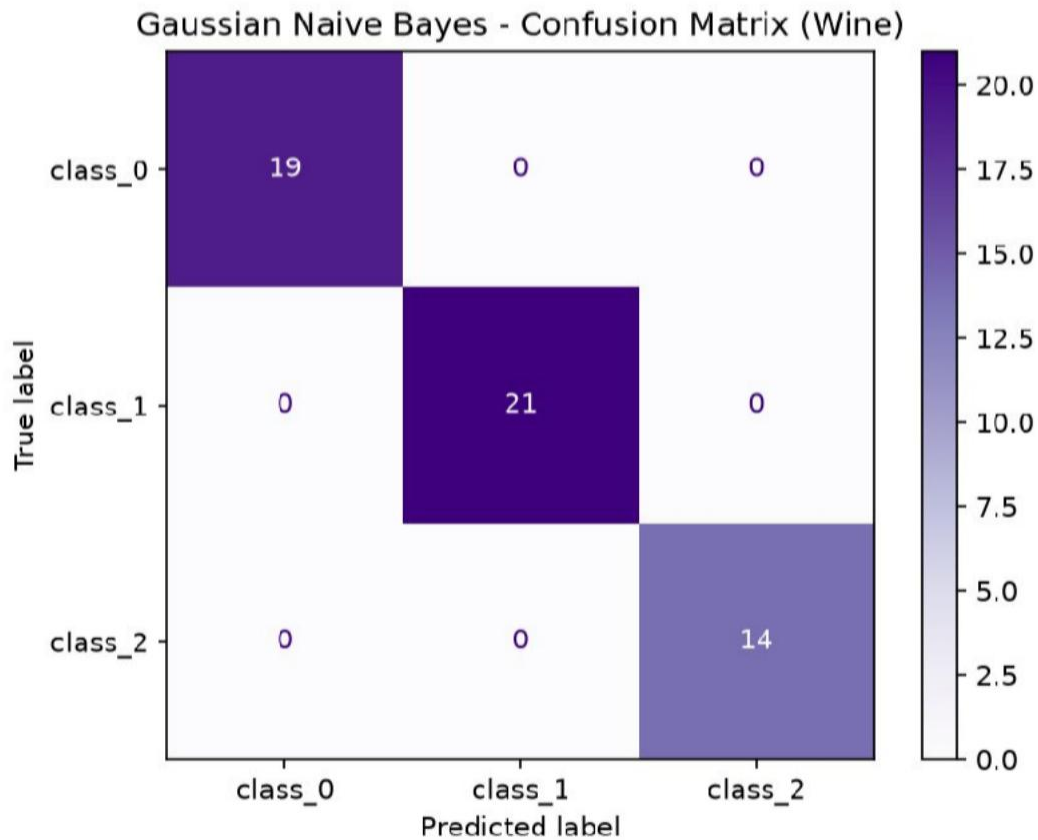
```
Classification Report:
```

	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	19
class_1	1.00	1.00	1.00	21
class_2	1.00	1.00	1.00	14
accuracy			1.00	54

macro avg	1.00	1.00	1.00	54
weighted avg	1.00	1.00	1.00	54

Prediction Probabilities (first 5):

	class_0	class_1	class_2
0	1.0	0.0000	0.0000
1	1.0	0.0000	0.0000
2	0.0	0.0018	0.9982
3	1.0	0.0000	0.0000
4	0.0	1.0000	0.0000



Result: Gaussian Naive Bayes classifier was applied on the Wine dataset. High accuracy was achieved. The confusion matrix showed minimal misclassifications.

PRACTICAL NO 12

Aim: To implement the Random Forest ensemble classifier and compare its performance with a single Decision Tree.

Objective: Train Random Forest with multiple trees, analyze feature importances, and compare with Decision Tree accuracy.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_wine

wine = load_wine()
X, y = wine.data, wine.target
feature_names = wine.feature_names

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_acc = accuracy_score(y_test, dt.predict(X_test))
print(f"Decision Tree Accuracy: {dt_acc:.4f}")

rf = RandomForestClassifier(n_estimators=100, max_depth=None, oob_score=True, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)
print(f"Random Forest Accuracy: {rf_acc:.4f}")
print(f"OOB Score: {rf.oob_score_:.4f}")
print("\nClassification Report:\n", classification_report(y_test, rf_pred,
target_names=wine.target_names))

estimators = [10, 20, 50, 100, 200]
accs = []
for n in estimators:
    rf_n = RandomForestClassifier(n_estimators=n, random_state=42)
    rf_n.fit(X_train, y_train)
    accs.append(accuracy_score(y_test, rf_n.predict(X_test)))

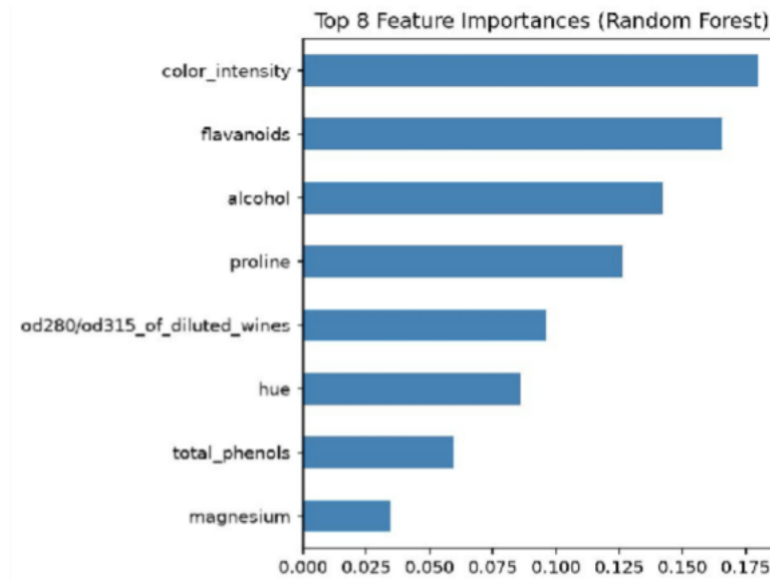
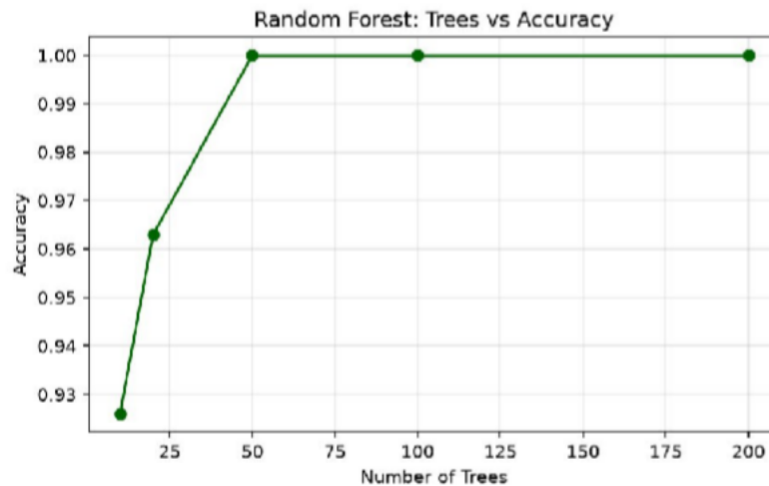
plt.figure(figsize=(7, 4))
plt.plot(estimators, accs, marker='o', color='darkgreen')
plt.xlabel('Number of Trees')
plt.ylabel('Accuracy')
plt.title('Random Forest: Trees vs Accuracy')
plt.grid(True, alpha=0.3)
plt.show()

importances = pd.Series(rf.feature_importances_, index=feature_names)
importances.nlargest(8).sort_values().plot(kind='barh', color='steelblue')
plt.title('Top 8 Feature Importances (Random Forest)')
plt.tight_layout()
plt.show()
```

Output:

Decision Tree Accuracy: 0.9630
Random Forest Accuracy: 1.0000
OOB Score: 0.9839

Classification Report:				
	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	19
class_1	1.00	1.00	1.00	21
class_2	1.00	1.00	1.00	14
accuracy			1.00	54
macro avg	1.00	1.00	1.00	54
weighted avg	1.00	1.00	1.00	54



Result: Random Forest outperformed single Decision Tree due to ensemble averaging. Feature importance plots revealed the most significant wine attributes.